AIAC-21.3

Name: Siri Vempati

Ht.no:2303A51309

Batch.no:05

Task-01:

Prompt:

Create a simple Expense Tracker application using HTML, CSS, and JavaScript.
The app should allow users to:

- Add an expense with name, amount, and category
- Display all expenses in a list
- Show total monthly expense

Then improve the application by:

- Adding input validation
- Displaying category-wise expense summary
- Refactoring the code into clean, reusable functions
- Making the UI simple and user-friendly

Code:

Ass-21.3 > t1.py > view_expenses

```python
# Improved Expense Tracker
expenses = []
def add_expense():
    name = input("Enter expense name: ")
    try:
        amount = float(input("Enter amount: "))
    except ValueError:
        print("Invalid amount! Please enter a number.")
        return
    category = input("Enter category: ")
    expenses.append({
        "name": name,
        "amount": amount,
        "category": category
    })
    print("Expense added successfully!")
def view_expenses():
    if not expenses:
        print("No expenses recorded.")
        return
    print("\nExpenses List:")
    for e in expenses:
        print(f"{e['name']} - ₹{e['amount']} - {e['category']}")
def monthly_summary():
    if not expenses:
        print("No expenses to summarize.")
        return
    total = sum(e["amount"] for e in expenses)
    print(f"\nTotal Monthly Expense: ₹{total}")
    category_total = {}
    for e in expenses:
        category_total[e["category"]] = category_total.get(e["category"], 0) + e["amount"]

    print("\nCategory-wise Summary:")
    for cat, amt in category_total.items():
        print(f"{cat}: ₹{amt}")
def main():
```

```python
def main():
    while True:
        print("\n1. Add Expense")
        print("2. View Expenses")
        print("3. Monthly Summary")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            add_expense()
        elif choice == "2":
            view_expenses()
        elif choice == "3":
            monthly_summary()
        elif choice == "4":
            print("Exiting program...")
            break
        else:
            print("Invalid choice")
main()
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/ve
empa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/t1.py

1. Add Expense
2. View Expenses
3. Monthly Summary
4. Exit
Enter choice: 1
Enter expense name: food
Enter amount: 1000
Enter category: explore
Expense added successfully!

1. Add Expense
2. View Expenses
3. Monthly Summary
4. Exit
Enter choice: 2

Expenses List:
3. Monthly Summary
4. Exit
Enter choice: 2

Expenses List:

Expenses List:
Expenses List:
food - ₹1000.0 - explore
```

Explanation:

This is an Expense Tracker to add, view, and summarize expenses.
It uses a menu-driven approach for user interaction.

Functions

Separate functions handle adding, viewing, and summarizing expenses.
This makes the code modular and easy to understand.

Validation

Input validation is done using try-except.
It prevents errors when invalid data is entered.

Conclusion

The program is simple, structured, and user-friendly.
It shows good coding practices using functions and validation.

Task-02

Prompt:

Create a Student Grade Calculator using HTML, CSS, and JavaScript.
The application should:

- Take marks of multiple subjects
- Calculate total, average, and percentage
- Assign grades (A, B, C, etc.) based on percentage

Then improve the program by:

- Refactoring code into functions
- Making the UI user-friendly
- Ensuring proper validation for marks input

Code:

Index.html

```
Ass-21.3 > task-02 > index.html > html
<!DOCTYPE html>
<html>
<head>
    <title>Grade Calculator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
    <h1> Grade Calculator</h1>

    <input type="number" id="m1" placeholder="Subject 1">
    <input type="number" id="m2" placeholder="Subject 2">
    <input type="number" id="m3" placeholder="Subject 3">

    <button onclick="calculate()">Calculate</button>

    <h3 id="result"></h3>
</div>

<script src="script.js"></script>
</body>
</html>
```

Style.css

Ass-21.3 > task-02 > # style.css > button

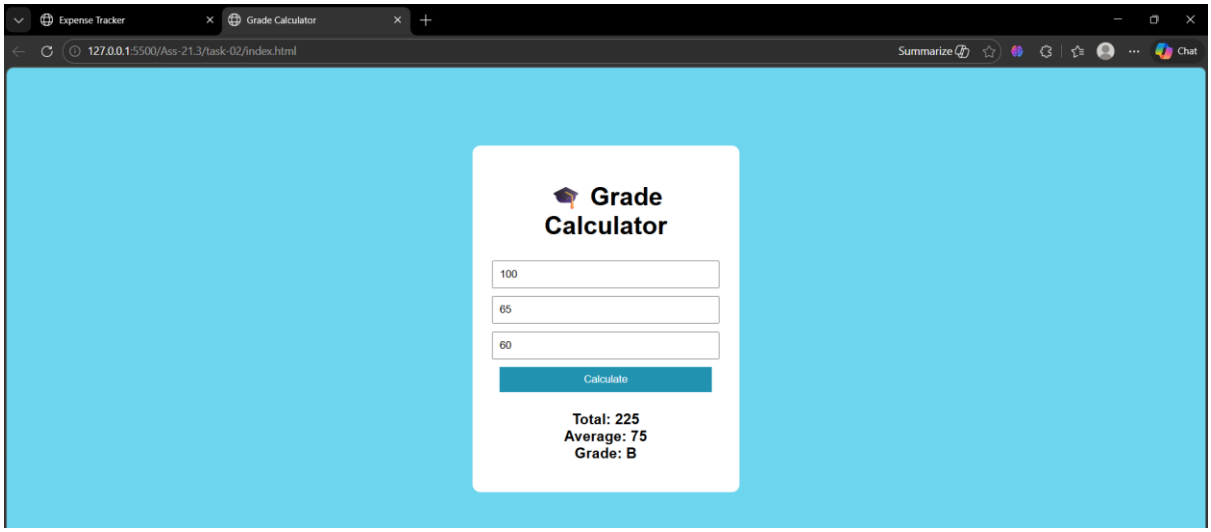
```css
body {
    font-family: Arial;
    background: #6dd5ed;
    text-align: center;
}

.container {
    background: white;
    padding: 20px;
    width: 300px;
    margin: auto;
    margin-top: 100px;
    border-radius: 10px;
}

input, button {
    width: 90%;
    margin: 5px;
    padding: 8px;
}

button {
    background: #2193b0;
    color: white;
    border: none;
}
```

Script.js

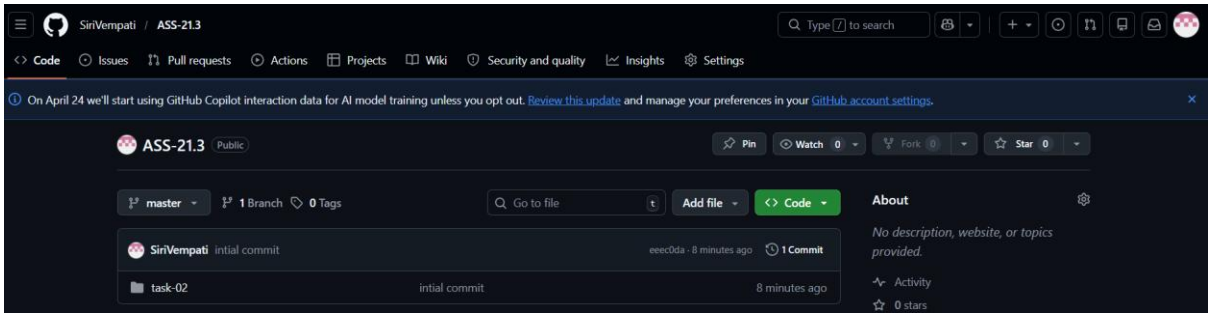
Ass-21.3 > task-02 > JS script.js > getGrade

```js
function calculate() {
    let m1 = parseFloat(document.getElementById("m1").value);
    let m2 = parseFloat(document.getElementById("m2").value);
    let m3 = parseFloat(document.getElementById("m3").value);

    if (isNaN(m1) || isNaN(m2) || isNaN(m3)) {
        alert("Enter all marks correctly!");
        return;
    }

    let total = m1 + m2 + m3;
    let avg = total / 3;
    let percentage = avg;

    let grade = getGrade(percentage);

    document.getElementById("result").innerHTML =
        `Total: ${total} <br> Average: ${avg} <br> Grade: ${grade}`;
}

function getGrade(p) {
    if (p >= 90) return "A";
    else if (p >= 75) return "B";
    else if (p >= 50) return "C";
    else return "Fail";
}
```

Output:




```
vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git branch task-02 feature
fatal: not a valid object name: 'feature'

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git branch task-02-feature

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git checkout task-02-feature
Switched to branch 'task-02-feature'

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (task-02-feature)
$ git checkout master
Switched to branch 'master'

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git merge task-02-feature
Already up to date.

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git remote add origin https://github.com/SiriVempati/ASS-21.3.git

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git push -u origin master
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.09 KiB | 1.09 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/SiriVempati/ASS-21.3.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```



Explanation

A Student Grade Calculator was developed to compute total, average, percentage, and grade.
AI helped generate and improve the application with better logic and structure.

AI Enhancements

AI added features like grade classification and input validation.
It also refactored the code into reusable functions for better readability.

Version Control Role

Git was used to manage changes and track versions of the project.
Branching allowed safe addition of new features without affecting the main code.

Collaboration Workflow

Feature branch was created for AI improvements and later merged into main.
Conflicts (if any) were resolved, simulating real-world team collaboration.

Conclusion

This task demonstrates how AI and Git together improve development workflow.
It ensures better code quality, organization, and collaborative coding practices.

Task-03

Prompt:

Create a Password Validation module using JavaScript.

The program should:

- Validate password length
- Check for at least one special character

Then generate meaningful Git commit messages describing:

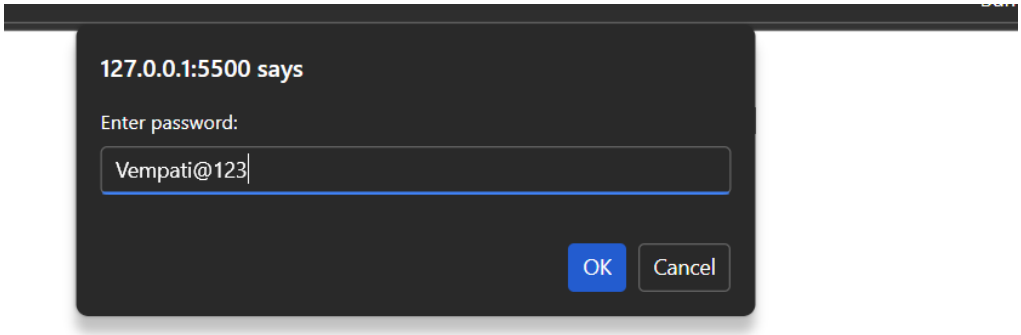
- Addition of validation rules
- Improvements in code structure

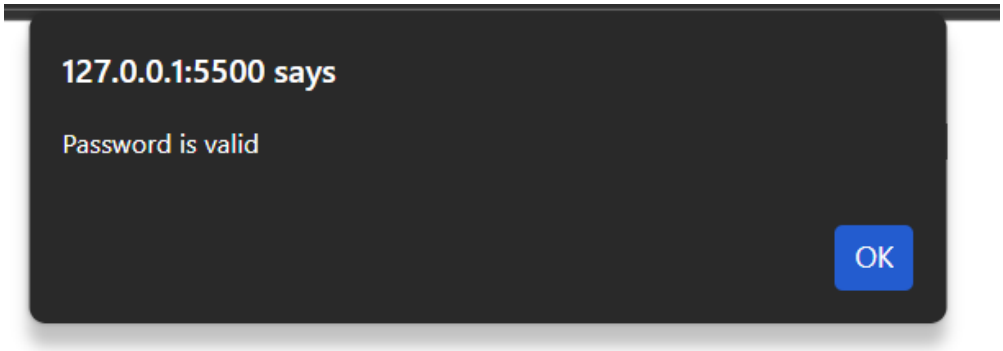
Also provide multiple commit message options and compare them with simple human-written messages.

Code:

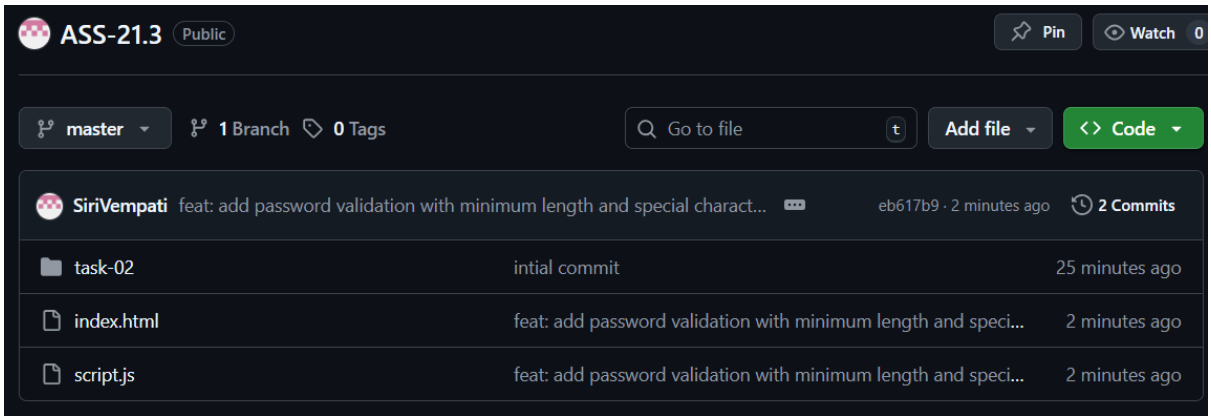
```
Ass-21.3 > JS script.js > validatePassword
function validatePassword(password) {

    // Minimum length check
    if (password.length < 8) {
        return "Password must be at least 8 characters long";
    }

    // Special character check
    let specialCharPattern = /[!@#$%^&*(),.?":{}|<>]/;
    if (!specialCharPattern.test(password)) {
        return "Password must contain at least one special character";
    }

    return "Password is valid";
}

// Example usage
let userPassword = prompt("Enter password:");
alert(validatePassword(userPassword));
```

Output:






```
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        t1.py

nothing added to commit but untracked files present (use "git add" to track)

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$ git log
commit eb617b9ca7a112ded96c469066b286a7cd8a9bc8 (HEAD -> master)
Author: SiriVempati <2303A51309@sru.edu.in>
Date:   Thu Apr 2 10:45:32 2026 +0530

    feat: add password validation with minimum length and special character requirement

commit eeec0da4fa9fc2fe713582b0693fee74332ef0a0 (origin/master, task-02-feature)
Author: SiriVempati <2303A51309@sru.edu.in>
Date:   Thu Apr 2 10:22:14 2026 +0530

    intial commit

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$ ^C

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$ git remote add origin https://github.com/SiriVempati/ASS-21.3.git
error: remote origin already exists.

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$ git push -u origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 732 bytes | 732.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/SiriVempati/ASS-21.3.git
   eeec0da..eb617b9  master -> master
branch 'master' set up to track 'origin/master'.

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$
```



Explanation

Code Functionality

The program validates a password based on rules.
It checks minimum length and presence of special characters.

AI Role in Commit Messages

AI generates **clear and descriptive commit messages**.
It explains what changes were made and why.

Importance of Good Commit Messages

Good messages help track changes easily.
They improve collaboration and project understanding.

Conclusion

AI-generated messages are more informative and professional.
They are better than simple human-written messages.

Task-04

Prompt:

Create a Quiz Management System in Python.

The program should:

- Display questions with options
- Accept user answers

Then improve it by:

- Adding score calculation
- Adding input validation
- Organizing code into functions

Simulate collaboration using Git branching and merging.

Code:

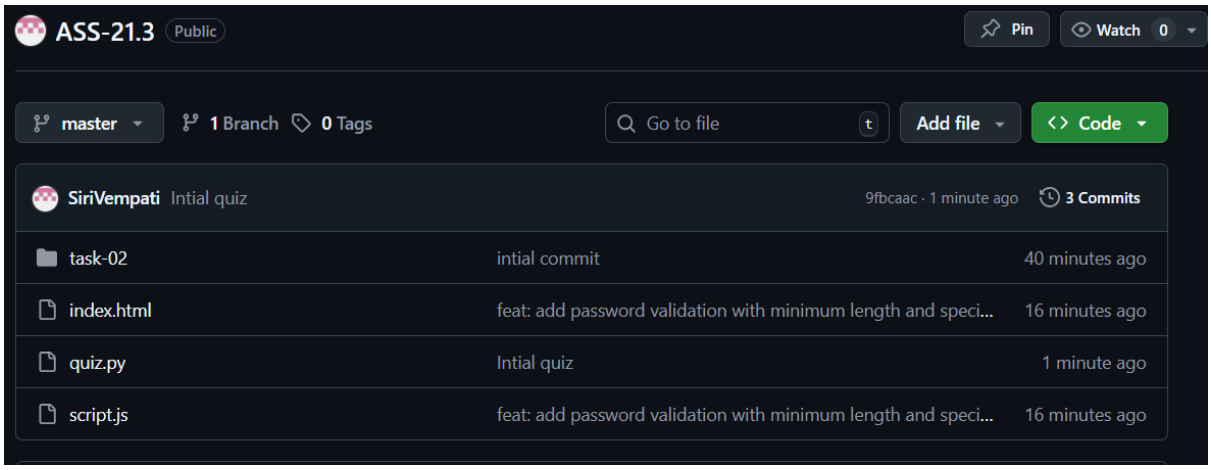
Ass-21.3 > quiz.py > start_quiz

```python
questions = [
    {
        "question": "Capital of India?",
        "options": ["Delhi", "Mumbai", "Chennai"],
        "answer": "Delhi"
    },
    {
        "question": "2 + 2 = ?",
        "options": ["3", "4", "5"],
        "answer": "4"
    }
]
def start_quiz():
    score = 0
    for q in questions:
        print("\n" + q["question"])
        for opt in q["options"]:
            print(opt)

        user_answer = input("Enter your answer: ")
        if user_answer not in q["options"]:
            print("Invalid option!")
            continue
        if user_answer == q["answer"]:
            score += 1
    print(f"\nYour Score: {score}/{len(questions)}")
start_quiz()
```

Output:

```
PS C:\Users\Vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/Ve
empa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/quiz.py

Capital of India?
Delhi
Mumbai
Chennai
Enter your answer: 1
Invalid option!

2 + 2 = ?
3
4
5
Enter your answer: 4

Your Score: 1/2
```

```
vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3 (master)
$ git add quiz.py

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git commit -m "Intial quiz "
[master 9fbcaac] Intial quiz
 1 file changed, 35 insertions(+)
 create mode 100644 quiz.py

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git branch feature-score

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (master)
$ git checkout feature-score
Switched to branch 'feature-score'

vempa@SIRIVEMPATI MINGW64 ~/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CO
DING/Ass-21.3 (feature-score)
$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 668 bytes | 668.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/SiriVempati/ASS-21.3.git
   eb617b9..9fbcaac  master -> master
branch 'master' set up to track 'origin/master'.
```



Explanation:

This program is a Quiz Management System that asks users questions and checks their answers. It stores questions, options, and correct answers using a list of dictionaries.

The start_quiz() function displays each question, takes user input, and validates whether the entered answer is among the given options. If the answer is correct, the score is increased.

At the end, the program calculates and displays the total score, making it a simple and structured example of using loops, functions, and validation in Python.